Lightweight Computer Vision Model for Real-Time Surface Defect Detection in Manufactured Components on Embedded/Edge Devices

Venkata Sai Anirudh Badveti
Reg. No. 240911244
School of Computer Engineering
MIT Manipal, MAHE
Manipal, India

Karanam Kartikeya
Reg. No. 240911648
School of Computer Engineering
MIT Manipal, MAHE
Manipal, India

Tanush Rakesh Desai
Reg. No. 240911542
School of Computer Engineering
MIT Manipal, MAHE
Manipal, India

Abstract: We evaluated a lightweight surface-defect detector across all 15 MVTec AD categories using a documented supervised split (seed 42): 4470 training, 398 validation and 486 held-out test images. An ImageNet-pretrained ResNet18 teacher and MobileNetV3-Small student jointly predict defect class and pixel mask; the student learns from teacher logits and maps. The student was exported to ONNX and statically quantized to INT8 with training-only calibration. Held-out image F1 was 94.1%, 92.1% and 78.9% for teacher, student FP32 and student INT8. Pixel AUROC was 0.965, 0.962 and 0.815; mask IoU was 0.610, 0.574 and 0.154. ONNX model files were 50.9, 9.0 and 2.6 MB. Saved images simulated camera frames on the laptop CPU; mean INT8 pipeline time was 59.2 ms, slower than the FP32 student. INT8 accuracy loss makes this export unsuitable without further tuning. A local MQTT test delivered 405 defect events. The split differs from the standard unsupervised MVTec protocol, so scores are not directly comparable. Physical camera, Raspberry Pi, Jetson and power measurements remain future work.

***Index Terms*—AIoT, edge AI, surface defect detection, MVTec AD, lightweight CNN, knowledge distillation, quantization, embedded systems, Industry 4.0**

I. Introduction

The Internet of Things (IoT) connects sensors, cameras, actuators and gateways into networked systems that monitor and act on the physical environment. In manufacturing this is the backbone of Industry 4.0: machines and inspection stations send data constantly to support monitoring and quality control. Streaming alone, though, only moves data around; it still must be turned into timely decisions. That is the role of Artificial Intelligence in IoT (AIoT), which adds machine learning (ML) and deep learning (DL) to the IoT stack so devices can classify, predict, detect anomalies and trigger actions without human intervention. Its value comes down to three properties: decisions are made near the data source, the system scales to large volumes of sensor and image data, and responses are fast enough for closed-loop control.

We target automated visual inspection on production lines. We catch surface defects such as scratches, cracks, dents, discoloration and contamination directly affect product reliability and cost, and manual inspection is slow, subjective and tough to scale. We can automate it using Deep-learning-based computer vision, but accurate CNNs are expensive to run. They need large memory, high-throughput processors and often GPUs, while the devices that actually sit on a line (single-board computers, Jetson-class modules, microcontroller vision boards) have tight limits on compute, memory, power and inference speed. Sending every image to the cloud is not really feasible as it adds latency, uses network bandwidth, raises concerns like data-privacy, and leaves the line dependent on an uninterrupted connection.

Existing challenges. New anomaly detection methods on MVTec AD get judged almost entirely by accuracy. They often rely on memory banks or large pretrained backbones and sometimes multiple models together. These setups hardly get checked on embedded hardware at all.

Lightweight classification backbones exist but pairing them with defect localization does not happen much. Compression techniques end up shown one at a time instead. Distillation or quantization or pruning each on its own. It seems like the full pipeline with accuracy and cost trade offs on a real edge device stays missing from most work.

Motivation. A factory that inspects parts on the device itself gets low latency, less network issues, offline operation and cheaper hardware. A systematic, measured route from a heavy baseline to a deployable model would make this kind of inspection very accessible to smaller sized manufacturers.

Problem statement. Given images of manufactured components, detect whether a surface defect is present and localize it, in real time, on a resource-constrained embedded device, while keeping accuracy close to that of a full-size CNN baseline.

Research objectives.

1) Build and evaluate a baseline CNN defect detector on MVTec AD.
2) Design a lightweight model using depthwise-separable convolutions, reduced channel width and knowledge distillation.

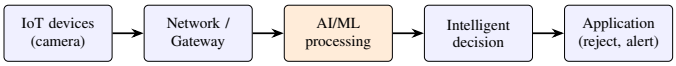


Fig. 1. General AIoT flow instantiated for visual inspection.

3) Compress the student with post-training INT8 quantization.
4) Benchmark teacher, student and INT8 inference on the available laptop; embedded-board timing and power tests remain future work.
5) Exercise event-only MQTT transport using a saved-image camera simulation.

Brief contribution. We implemented a supervised 15-category dataset split, trained a class-and-mask ResNet18 teacher and a distilled MobileNetV3-Small student, and verified a static INT8 ONNX export. We measured image and pixel quality on held-out images, model size, laptop GPU model latency, laptop CPU pipeline latency and local MQTT delivery. The proposed Raspberry Pi or Jetson deployment in Fig. 2 was not physically evaluated. The code, split manifest, logs and model outputs accompany this report. Per-category counts and test metrics are provided in the supplementary table. A common ONNX CPU runtime times all three model files, while PyTorch GPU tests measure the teacher and student forward passes. The source-image simulation includes loading and postprocessing. These measurements define what can be replicated on the available laptop.

II. Literature Survey

We studied four representative works on industrial image anomaly and defect detection. The first is the base paper, which introduces the benchmark; the other three are methods evaluated on it. Supporting work on efficient networks and compression is covered in Section III.

A. Base paper: The MVTec Anomaly Detection Dataset [1]

Bergmann et al. [1] address the lack of a realistic benchmark for unsupervised anomaly detection and localization in industrial inspection. The work has no IoT architecture; it frames inspection as an offline image problem. The dataset has 15 object and texture categories (including bottle, cable, capsule, metal nut, screw, tile and wood), defect-free training images, and test images covering more than 70 defect types with pixel-precise ground truth. The models are trained using only defect-free images and are assessed on mixed test sets with the help of both image-level and pixel-level (localization) metrics. The authors look at a variety of methods, including convolutional autoencoders and generative models as well as feature-based and classical methods. The benchmark is realistic, high-resolution and has now become the standard. However, for our purposes it has the drawback that it does not mention anything regarding computational cost, memory usage or the possibility of deploying the systems on embedded hardware, and the most effective methods that it compares were not developed with on-device use in mind.

B. PatchCore [2]

PatchCore [2] seeks to achieve the highest possible recall of defective samples in the scenario of cold-start anomaly detection, where only a few or no defective examples are available. It obtains locally aware mid-level patch features from a CNN that has been pretrained on ImageNet, keeps these features in a memory bank, reduces the size of the bank using coreset subsampling, and then evaluates test patches based on their nearest-neighbor distance. The authors achieved an image-level AUROC of about 99.1% on MVTec AD with only a small amount of training data, the evaluation being carried out offline on a GPU. However, for use on edge devices the drawback is that it still requires a large pretrained backbone and a memory bank, and the nearest-neighbor search causes latency and memory pressure that a small device cannot handle.

C. PaDiM [3]

The PaDiM [3] method is able to detect and localise anomalies without having to store or search through a large training set when it is being tested. It makes use of a pretrained CNN to produce patch embeddings, with each patch position being represented by a multivariate Gaussian, and a test patch is then evaluated based on its Mahalanobis distance to the Gaussian associated with that position. It performs well in terms of localisation and does not require any training of the backbone network. The drawback is that the amount of covariance data stored per position increases as both the feature resolution and the embedding size grow, and the approach is completely dependent on a large pretrained network, which means that it is not a compact, quantised, and embedded-ready model.

D. Student-teacher discriminative embeddings [4]

Bergmann et al. [4] aim at achieving accurate anomaly segmentation by using a more compact learned component. They distill a teacher network, which has been pre-trained on natural images, into smaller student descriptor networks that are then trained on normal data, and the anomaly map is obtained from the difference between an ensemble of the student networks and the teacher. This demonstrates that distillation can transfer local feature knowledge to smaller networks. Yet, inference still requires both the ensemble of students and the pre-trained teacher, and the authors do not examine edge latency, memory usage or quantization behaviour.

E. Comparative analysis and research gap

Table I compares the surveyed works. All four are benchmarked on accuracy and localization quality on server-class hardware. None reports on-device latency, memory footprint or power draw, none applies quantization or pruning, and none describes how the detector would sit inside a live IoT workflow. Meanwhile, efficient backbones such as MobileNet [5] and EfficientNet [6], distillation [7] and integer-arithmetic quantization [8] are well established in general vision, but they have not been combined and measured as a single compression pipeline for defect detection and localization on embedded hardware.

Research gap: there is no systematic, hardware-measured pipeline that takes an industrial defect detector from a heavy baseline to a distilled, quantized lightweight model, retains localization, and integrates it into an edge-to-cloud AIoT workflow with event-only communication.

TABLE I
COMPARATIVE ANALYSIS OF SURVEYED WORKS AND THE PROPOSED APPROACH

Work	AI technique	Application	Advantages	Limitations
MVTec AD paper [1] (base)	Benchmark of autoencoder, generative and feature-based methods	Industrial inspection benchmark	Realistic data, pixel-level ground truth, standard protocol	No cost, memory or edge evaluation
PatchCore [2]	Patch-feature memory bank with coreset, nearest neighbor	Anomaly detection and localization	Very high image-level AUROC, few training images	Large backbone, memory bank, search latency
PaDiM [3]	Per-position Gaussian on CNN patch embeddings	Anomaly detection and localization	Simple, good localization	Covariance storage, heavy backbone, no compression
Student-teacher [4]	Distilled student ensemble vs. pretrained teacher	Anomaly segmentation	Distillation transfers local features	Ensemble and teacher needed at inference
Proposed	Distilled MobileNetV3-Small + INT8	Edge defect detection with localization	2.6 MB INT8; laptop latency	Physical edge deployment remains open

III. PROPOSED METHODOLOGY

A. What is new compared with existing systems

This study joins supervised class and mask prediction, teacher-to-student distillation, static INT8 quantization and event-only MQTT transport in one reproducible pipeline. We measure each model stage on the available laptop and report both image and pixel quality. Fig. 2 shows the intended edge deployment; its camera, actuator and cloud path are design targets rather than completed physical tests. The saved-image simulation exercises frame processing and local event delivery without claiming embedded-board timing.

B. Architecture and workflow

Fig. 2 shows the end-to-end design: an offline stage that builds and compresses the model, and a runtime stage on the embedded device.

C. Data collection and preprocessing

The MVTec AD dataset [1] provides 15 categories of normal and defective industrial images, with pixel masks for defects. The original training/good images stay in the training split. Because this project uses supervised defect examples, every original test subtype is shuffled with a SHA256-derived seed from seed 42. Floor(n/2) images enter training, floor(n/4) validation, and the remainder are held out for testing. Thus, train, validation and test paths are disjoint within every category and defect subtype. This custom protocol differs from the normal-only MVTec benchmark; published unsupervised scores cannot be compared directly. The archive contains 5354 images. The exact counts for this split are 3860 normal and 610 defect training images, 113 normal and 285 defect validation images, and 123 normal and 363 defect held-out images. Checkpoints and class/mask thresholds use validation only; INT8 calibration uses training only.

Inputs are resized to 224 x 224 RGB and normalized with ImageNet statistics. Masks use nearest-neighbor resize. Training uses random horizontal flips and a class-balanced replacement sampler; no synthetic defects or brightness/rotation augmentation were used.

D. Baseline model

The baseline is an ImageNet-pretrained ResNet18 encoder with a binary classification head and a skip-connected mask decoder. Both heads are fine-tuned on the supervised split. Pixel masks are trained directly, so the output is a segmentation prediction rather than a Grad-CAM explanation. It establishes classification, localization, model-size and latency reference values.

E. Lightweight student model

The student adopts the efficient-architecture concepts presented in MobileNet and EfficientNet-Lite [5], [6]. Instead of using a standard $K \times K$ convolution with C_{in} input and C_{out} output channels, a depthwise $K \times K$ convolution is used followed by a 1×1 pointwise convolution. The ratio of computational cost to that of a standard convolution is:

$$\frac{1}{C_{out}} + \frac{1}{K^2} \tag{1}$$

This implementation uses MobileNetV3-Small depthwise-separable features with a skip-connected mask decoder. No separate width multiplier, pruning or synthetic-defect generator was applied. The teacher is absent at student inference time.

F. Knowledge distillation

The student minimizes supervised class BCE, weighted mask BCE and Dice, plus softened teacher class BCE at T = 2 and map MSE. The implemented objective is:
L = BCEc + 0.5 BCEm + 0.5 Dice + 0.3 BCEkd + 0.2 MSEmap.
Teacher outputs are detached; the student uses teacher outputs only during training. Validation loss selects the checkpoint.

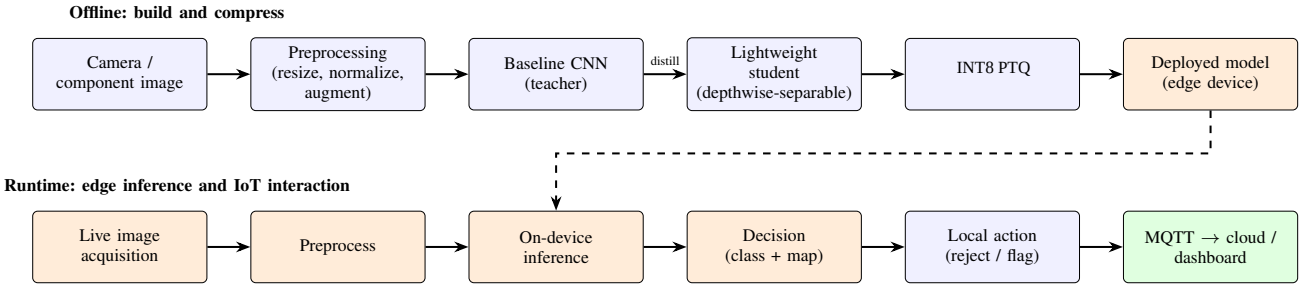


Fig. 2. Proposed edge-first AIoT methodology. Top: offline training and compression. Bottom: runtime inspection on the embedded device with event-only cloud communication. Orange: edge; green: cloud.

The class term uses sigmoid-scaled logits at temperature 2. Map MSE compares student and teacher mask probabilities. The training recipe uses AdamW with a 1e-4 learning rate, balanced image sampling and early stopping. No test image enters training or model selection.

G. Quantization and compression

After training, the student is exported to FP32 ONNX and converted by static post-training quantization to signed INT8 QDQ format with per-channel weights. Each of the 15 categories contributes four normal and four defect training images to calibration. The ONNX graph is checked for quantize and dequantize nodes and INT8 initializers; its FP32 output is compared numerically with PyTorch. The INT8 model is then evaluated on the same held-out test split, with thresholds selected on validation. Model file size and CPU inference time are measured. QDQ may add overhead, so speedup is measured rather than assumed. Pruning and quantization-aware fine-tuning were not performed.

H. The decision-making process and its interaction with the IoT

For the tested runtime, saved held-out images act as virtual camera frames. Each file is loaded, resized and normalized, then the model predicts a defect probability and a pixel map. Class and mask thresholds are selected on validation by image F1 and aggregate mask IoU, respectively. A bounding box is extracted from mask pixels above the mask threshold. Only frames whose class score exceeds the class threshold produce JSON defect events; normal frames generate no event. The CPU pipeline timer includes image load, preprocessing, model forward pass and postprocessing. A separate model-only benchmark measures GPU inference at batch size one after warm-up. An MQTT QoS 1 test sends the event JSON to a local broker and checks that the subscriber receives identical messages. This demonstrates event-only transport in a laptop simulation. The proposed camera, actuator, cloud dashboard, Raspberry Pi and Jetson in Fig. 2 were not used. Their latency, memory use and power draw are not inferred from the laptop results.

The event log and MQTT check are reproducible from the packaged scripts and held-out image paths.

I. Performance evaluation methodology

All three stages use one custom supervised split and the same held-out test images. We report accuracy, precision, recall, F1 and image AUROC for defect classification, plus pixel AUROC and mask IoU against the ground-truth masks. Validation chooses both thresholds before test evaluation. We report parameter count, estimated convolution/linear MACs and model file size. Speed measurements include mean and 95th percentile GPU model-only latency after 30 warm-ups and 150 batch-one trials, and CPU end-to-end saved-image latency over the test set. The CPU pipeline measurements include disk loading and preprocessing. We compare the FP32 teacher, FP32 student and INT8 ONNX student under the measured laptop conditions. Physical edge-device and power measurements require hardware and are excluded from numeric comparisons.

IV. Implementation and Experimental Setup

Experiments ran on a Ryzen 7 7840HS laptop with 16 GB RAM and an NVIDIA RTX 4060 Laptop GPU (8 GB VRAM). Python 3.13, PyTorch 2.11.0+cu128 and ONNX Runtime 1.31.1 were used. Both encoders started from ImageNet weights and used 224 x 224 inputs. AdamW used learning rate 1e-4 and weight decay 1e-4; training used batch size 8, up to 8 epochs, patience 3 and seed 42. The student used T = 2, class distillation weight 0.3 and map weight 0.2. The exact split, training logs, checkpoint files and validation thresholds are packaged with this report. Student training reused a lossless 224 x 224 cache; pipeline timing read original full-size files. Laptop CPU and GPU timings refer to different scopes and should not be interchanged.

V. Results and Comparison

Tables II and III report held-out quality and laptop CPU pipeline cost. ResNet18, student FP32 and student INT8 image F1 scores were 94.1%, 92.1% and 78.9%. Their pixel IoUs were 0.610, 0.574 and 0.154. INT8 shrank the file but sharply reduced quality and did not speed the CPU pipeline. The custom split prevents direct comparison with standard MVTec scores.

TABLE II
CUSTOM SUPERVISED 15-CATEGORY TEST

Model	Accuracy	Precision	Recall	F1
Baseline CNN (FP32)	91.2	94.0	94.2	94.1
Lightweight (FP32)	87.7	87.8	97.0	92.1
Lightweight (INT8)	66.7	74.8	83.5	78.9

TABLE III
LAPTOP CPU SAVED-IMAGE PIPELINE

Model	Size (MB)	Params (M)	Latency (ms)	FPS
Baseline CNN (FP32)	50.9	12.74	68.6	14.6
Lightweight (FP32)	9.0	2.25	56.1	17.8
Lightweight (INT8)	2.6	2.25	59.2	16.9

Red: physical Pi/Jetson and power N/E. Black: measured.

VI. Conclusion

Across all 15 MVTec AD categories and 486 held-out images, the ResNet18 teacher reached 94.1% image F1 and 0.610 pixel IoU. The distilled MobileNetV3-Small reached 92.1% F1 and 0.574 IoU; its INT8 version reached 78.9% F1 and 0.154 IoU. The INT8 ONNX file was 2.6 MB versus 9.0 MB for FP32 ONNX. Laptop CPU saved-image pipeline times were 68.6, 56.1 and 59.2 ms per image. The student FP32 saved 12.5 ms relative to the teacher, but INT8 lost 13.2 F1 points and added 3.1 ms relative to FP32 student inference. Static PTQ is therefore not a suitable deployment choice here without better calibration or quantization-aware training. A local MQTT QoS 1 test delivered all 405 flagged events from virtual camera frames. These findings support a smaller software inspection pipeline, but no production line, physical camera, embedded board, actuator, cloud dashboard or power meter was tested. The custom supervised split is not directly comparable with published normal-only MVTec results. Future work is physical deployment, power measurement, robustness to lighting and vibration, and testing additional industrial datasets.

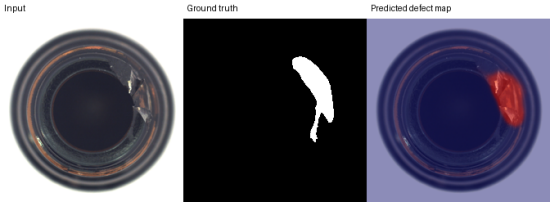


Fig. 3. Held-out bottle image, ground-truth mask and distilled student's predicted defect map. The map is a trained segmentation output; other category examples are in the experiment package.

References

[1] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *Int. J. Comput. Vis.*, vol. 129, no. 4, pp. 1038–1059, 2021.

[2] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 14318–14328.

[3] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Pattern Recognition. ICPR Int. Workshops and Challenges*, 2021, pp. 475–489.

[4] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "Uninformed students: Student–teacher anomaly detection with discriminative latent embeddings," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 4183–4192.

[5] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017.

[6] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[7] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv:1503.02531*, 2015.

[8] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[10] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.

[11] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

SUPPLEMENTARY EXPERIMENTAL DETAILS

TABLE IV

HELD-OUT RESULTS BY MVTEC AD CATEGORY

Category	Train	Val	Test	F1 T	F1 S	F1 Q	IoU T	IoU S	IoU Q
bottle	250	20	22	90.3	97.0	87.2	0.421	0.460	0.068
cable	298	34	42	98.2	96.2	78.3	0.478	0.427	0.075
capsule	283	30	38	92.1	89.9	89.9	0.267	0.300	0.052
carpet	336	27	34	100.0	98.2	84.7	0.633	0.577	0.116
grid	301	17	24	94.1	91.4	51.9	0.284	0.249	0.005
hazelnut	445	26	30	100.0	88.9	80.0	0.367	0.245	0.223
leather	305	28	36	100.0	100.0	57.8	0.595	0.510	0.062
metal_nut	276	26	33	96.2	92.0	90.0	0.706	0.788	0.340
pill	348	39	47	92.0	92.0	92.0	0.761	0.741	0.235
screw	398	38	44	89.2	85.7	85.7	0.143	0.084	0.024
tile	287	27	33	93.3	97.9	78.3	0.689	0.581	0.227
toothbrush	81	10	11	80.0	84.2	84.2	0.481	0.562	0.198
transistor	263	23	27	77.4	63.2	61.5	0.646	0.478	0.266
wood	285	17	24	90.9	97.1	87.8	0.532	0.540	0.120
zipper	314	36	41	100.0	100.0	34.1	0.659	0.586	0.015

T = ResNet18 teacher; S = MobileNetV3-Small FP32; Q = student INT8. F1 is image-level; IoU is pixel-level.

TABLE V

MODEL COST AND LAPTOP CPU LATENCY (ONNX RUNTIME)

Model	Params M	MACs G	ONNX MB	Model ms	Full ms	P95 ms
Teacher FP32	12.74	2.266	50.9	16.4	68.6	87.3
Student FP32	2.25	0.369	9.0	4.6	56.1	73.7
Student INT8	2.25	0.369	2.6	7.6	59.2	76.0

Experimental protocol. All 5354 source images are assigned once. Original train/good stays in training; each test subtype is shuffled with a stable seed and divided 50/25/remainder into train/validation/test. The split audit found no duplicate paths or missing masks. Validation selected checkpoints and thresholds; calibration used 120 training images, four normal and four defective per category. The INT8 graph contained 119 QuantizeLinear, 243 DequantizeLinear and 243 INT8 initializer tensors. PyTorch/FP32 ONNX parity was checked numerically. Table V uses the same ONNX CPU runtime for all three model files; its full time includes saved-image loading, resize and postprocessing.

Measurement scope. Table III reports CPU saved-image throughput, not camera or factory throughput. GPU model-only batch-one timing was 8.5 ms for the teacher and 14.3 ms for the student; it excludes file loading and preprocessing. MACs are THOP estimates for convolution and linear layers and exclude interpolation. MQTT received 405/405 QoS 1 events from the saved-image simulation on localhost. The per-category values above show variation hidden by aggregate scores. Physical Raspberry Pi and Jetson performance, memory, power, camera timing, network latency and local actuation remain unmeasured; the red note on page 5 marks them as planned. No numeric estimates are presented for those systems.